NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

Centre for Professional and Continuing Education

(SCTP) Advanced Professional Certificate

Data Science and AI



Interactive html: https://su-ntu-ctp.github.io/6m-data-3.6-Time-Series-Forecasting/

L06
Time Series
Forecasting

Sarah's week of forecasting NorthStar revenue

Copyright © 2026 Nanyang Technological University. All rights reserved.

Text

SLIDO

Interactive html: https://su-ntu-ctp.github.io/6m-data-3.6-Time-Series-Forecasting/

Forecasting is generally treated as supervised learning, with one interesting twist.

Why it's supervised. There's a clear target: the future value you want to predict, like next month's sales.

Where unsupervised methods appear. Some time-series tasks are unsupervised. Decomposition (STL) just separates patterns with no target, while anomaly detection flags unusual points (e.g. large residuals) without labels.

But forecasting itself, predicting a known kind of future value, is supervised.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Where We Left Off

L05 → L06 — from cross-sectional to time-ordered

Marcus, end of L05:

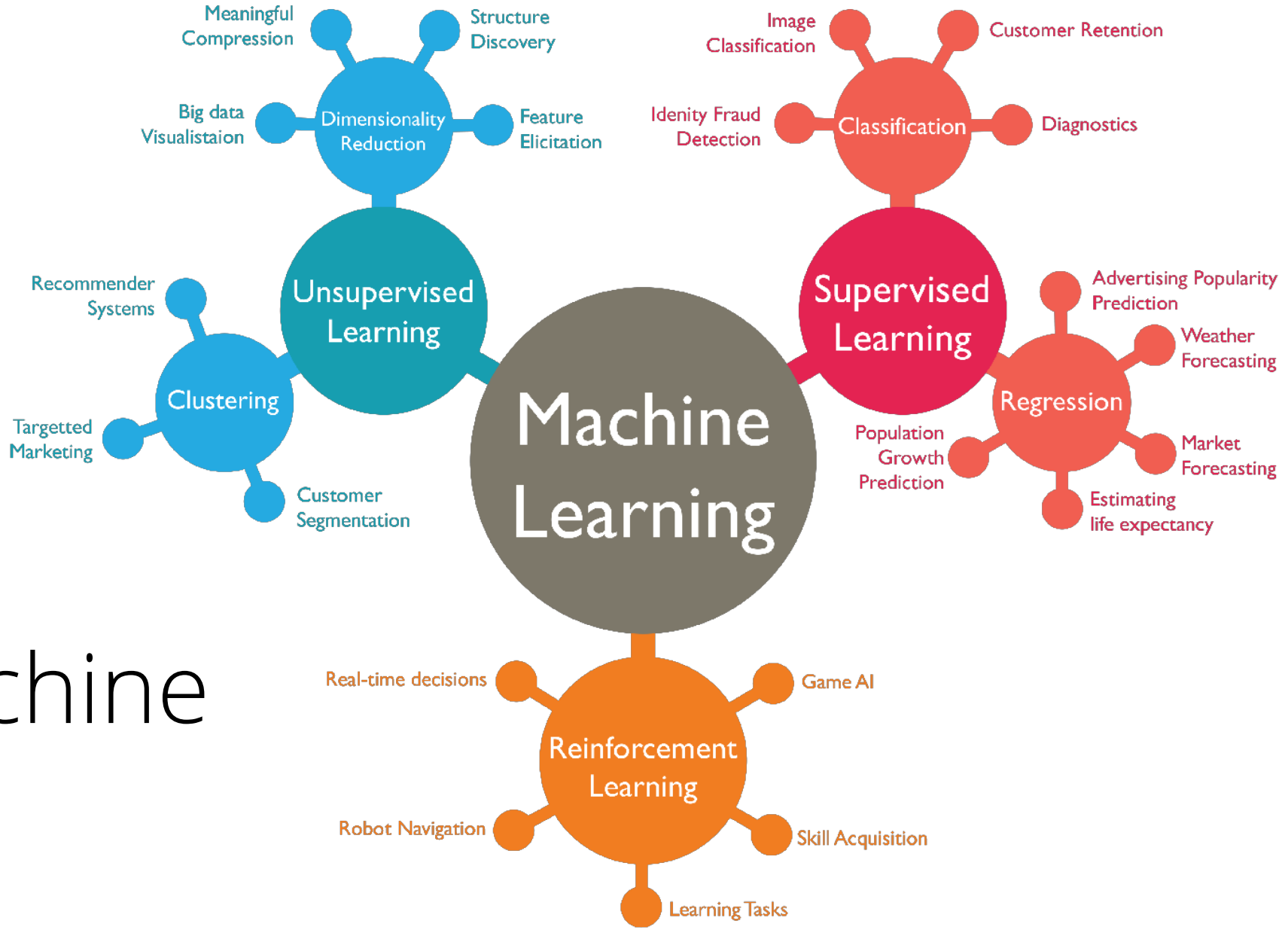
"Sales are SEASONAL. We need to know quarterly revenue 90 days ahead so ops can plan inventory.
Can you forecast?"

The fundamental shift

🗓️ **Data is TIME-ORDERED**	Order matters. Today's value depends on yesterday's.
🔮 **The future hasn't happened**	Never use data from after the prediction date.
🚫 **k-fold cross-val is FORBIDDEN**	Would leak the future into past training folds.
📈 **New toolkit**	Decomposition, ETS, lag features — time-aware methods

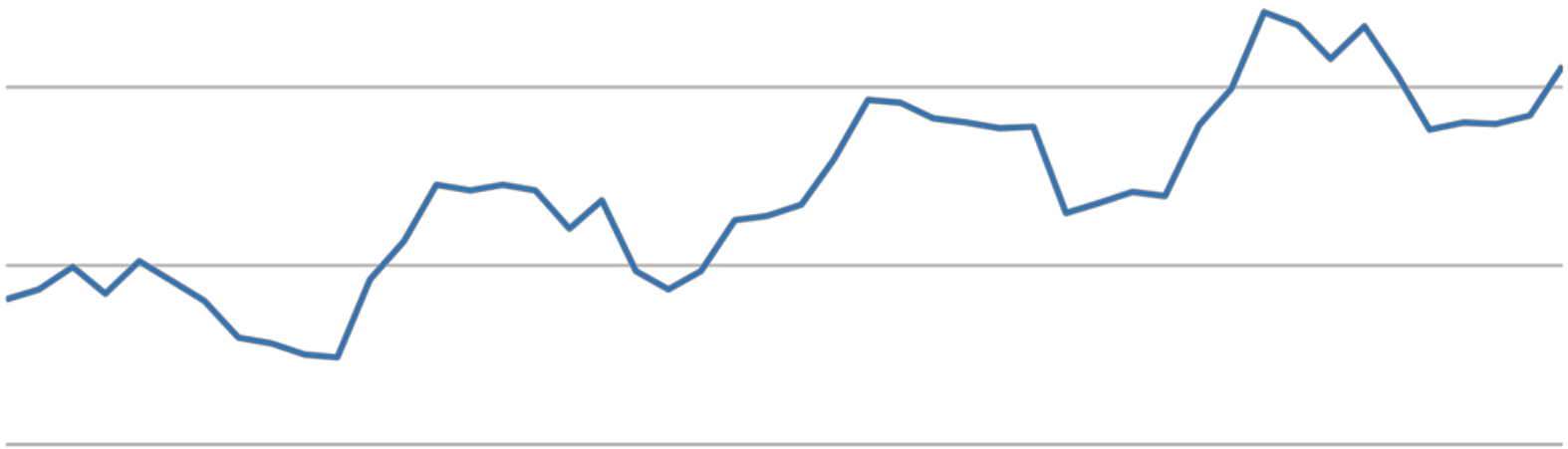
Copyright © 2026 Nanyang Technological University. All rights reserved.

Types of Machine Learning



Copyright © 2026 Nanyang Technological University. All rights reserved.

A time series is a sequence of observations ordered in time



Monthly sales, 4 years (illustrative). Each point is one value at one timestamp.

Order matters

Shuffle the rows and you destroy the information.

Regular spacing

Hourly, daily, monthly: the frequency is part of the data.

Today depends on yesterday

Observations are not independent, unlike most ML datasets.

Copyright © 2026 Nanyang Technological University. All rights reserved.

What are some forecasts you make in your day to day life?

Copyright © 2026 Nanyang Technological University. All rights reserved.

Demand Forecasting in the real world

RETAIL

Walmart

500M

item–store forecasts every week

Forecasts: 100k items × 4,700 stores

Method: XGBoost + state-space, on GPUs

Used for: store replenishment

E-COMMERCE

Amazon

11–15 pts

lower error at Decathlon with Chronos-2

Forecasts: demand across huge catalogues

Method: DeepAR and Chronos models

Used for: inventory, 12–52 weeks out

RIDE-HAILING

Uber

2–18%

more accurate than its old model

Forecasts: rides on holidays and events

Method: LSTM with weather inputs

Used for: driver supply and budgets

ENTERPRISE PLANNING

SAP IBP

ETS

exponential smoothing, built in

Forecasts: product demand for planners

Method: single, triple and automated ETS

Used for: sales and operations planning

An LSTM (Long Short-Term Memory) is a specialized type of Recurrent Neural Network (RNN) designed to process sequential data and remember long-term dependencies.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Framing the problem

Question	Example answer: grocery chain
What exactly are we forecasting?	Daily units of each product at each store
How far ahead? (horizon)	Next 14 days, refreshed every Monday
At what granularity, frequency?	Daily per store, not hourly
What is known in advance? Exogenous factors?	Promotions and public holidays yes; weather no
What does an error cost?	Stock-outs lose sales; over-ordering wastes fresh food

Lag is a value from an earlier time period, used as a feature to help predict the present or future. It mainly comes up with time series data, where rows are ordered by time (days, weeks, months).

E.g.:
Lag 1 = the value 1 period earlier
Lag 2 = the value 2 periods earlier

The recent past is often the best predictor of the near future.

Lag features come in: past values become the inputs

Vocabulary

Frequency — spacing between points (daily, weekly)

Horizon — how far ahead we forecast (next 28 days)

Lag — a past value, e.g. y one step back

Exogenous — outside drivers: price, promos, weather

Copyright © 2026 Nanyang Technological University. All rights reserved.

The Time-Series Trinity

Every series = Trend + Seasonality + Residual

Trend

Long-term direction.
Growth, decline, flat.
Smooth, slow-moving.

Seasonality

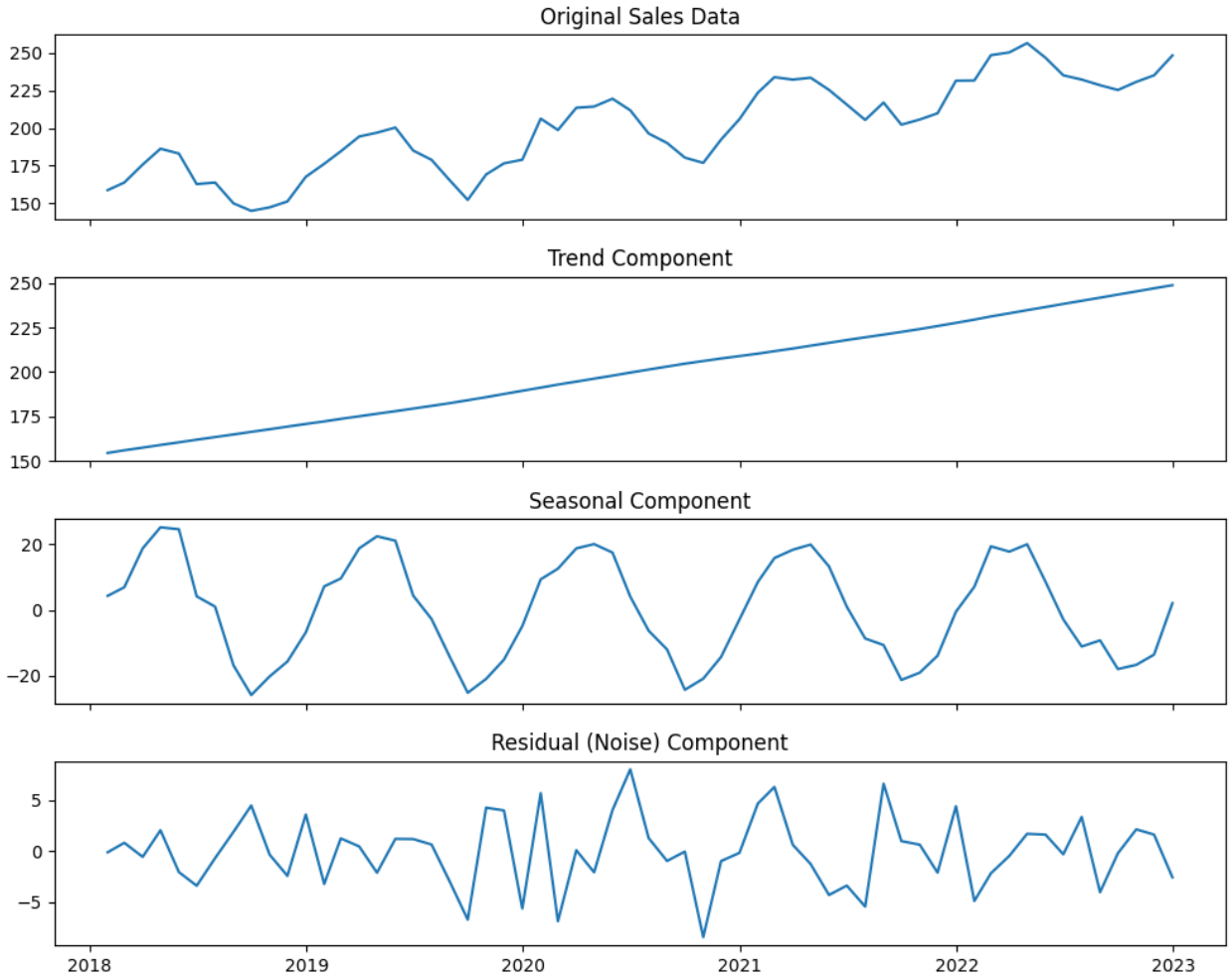
Seasonality is a repeating pattern that happens at regular, predictable intervals

Repeating cycles.
Annual, monthly, weekly, daily.
Predictable.

Residual

What's left — noise + unexplained.
Should look like random fluctuation.

Residual is what's left over after removing the trend and seasonality: the irregular, unexplained part



Large residuals are a common way to spot anomalies, such as an outage or an unexpected surge.

Copyright © 2026 Nanyang Technological University. All rights reserved.

STL Decomposition

The robust default — captures one seasonality at a time

Seasonal-Trend decomposition using LOESS. It's a popular method for splitting a time series into the three parts: treand, seasonal and residual.

The decomposition equation

$$y(t) = \text{trend}(t) + \text{seasonal}(t) + \text{residual}(t)$$

```
from statsmodels.tsa.seasonal import STL

stl = STL(y, period=7, robust=True).fit()
stl.trend       # the slow-moving direction
stl.seasonal    # the 7-day repeating pattern
stl.resid       # what's left — should look like noise
```

It also prepares data for forecasting, for example by modelling the trend and seasonality separately or by deseasonalising the series (observed − seasonal) to see the underlying movement more clearly.

STL handles ONE seasonality. For multi-seasonal data, use MSTL or capture other seasonalities via lag features.

Copyright © 2026 Nanyang Technological University. All rights reserved.

The purpose of decomposition itself. Splitting a series into parts helps you:

Understand what's happening. Is the business really growing (trend), or does it just look busy because it's peak season (seasonality)?

Make fair comparisons. December sales are always high. Removing seasonality (deseasonalising) shows whether this December was actually better than usual.

Spot anomalies. Unusually large residuals flag one-off events, like a system outage or a viral promotion.

Help forecasting. You can project the trend forward and re-add the seasonal pattern to predict future months.

Time Series Decomposition

2 key methods:

If the seasonal boost comes from a fixed outside source or a limit, it tends to stay the roughly the same size →additive.
If the seasonal boost comes from your growing customer base, it grows with you → multiplicative.

Additive

$$Y = T + S + R$$

- Consistent change on numerical scale
- Used when seasonal variation is independent of the trend.

Additive: swings stay the same height. The bump doesn't care how big the business is

Multiplicative

$$Y = T * S * R$$

- Change is on a relative scale
- Used when seasonal variation is proportional to the trend.

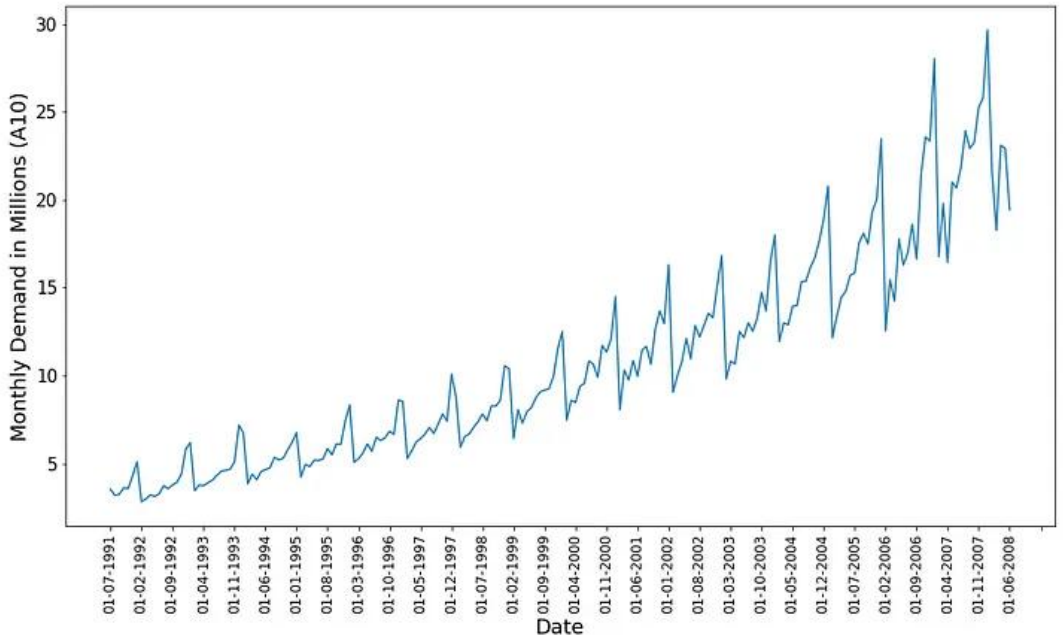
Multiplicative: swings grow with the trend. seasonal effects usually scale with the size of the business.

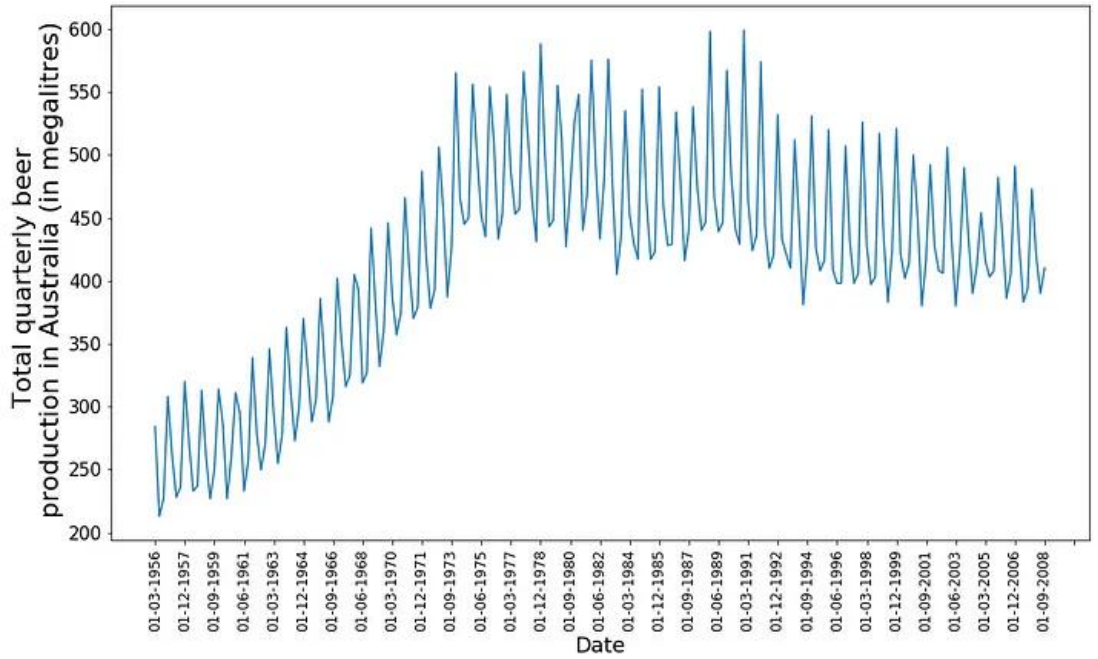
Key Considerations

- ✓ Use multiplicative when seasonal variation grows as the trend increases (peaks and troughs get bigger).
- ✓ Possible to change a multiplicative model to additive via log transformation / Box-Cox Transformation.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Additive or Multiplicative?



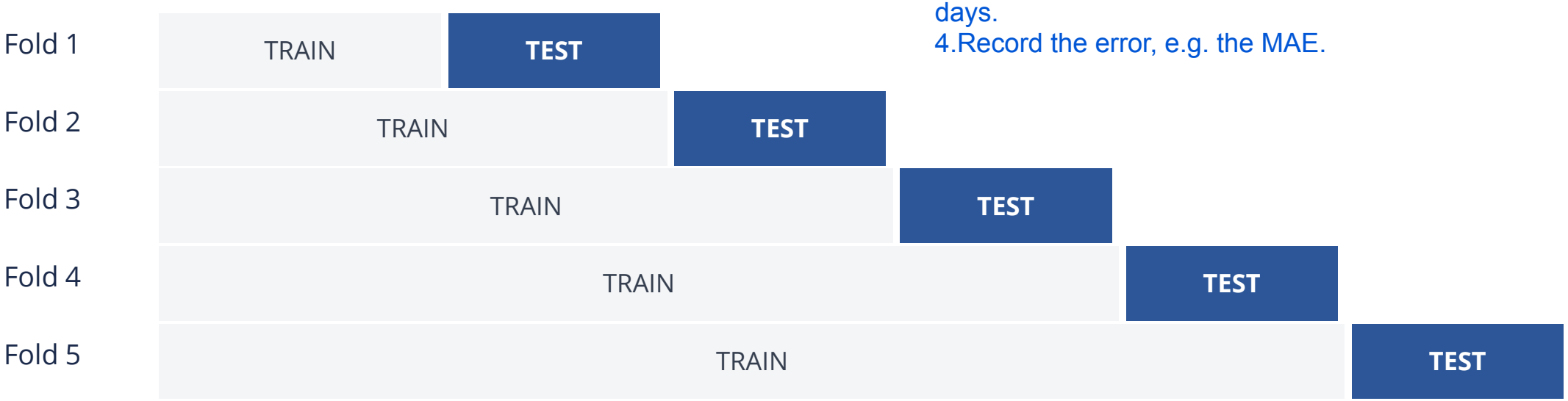


Copyright © 2026 Nanyang Technological University. All rights reserved.

Time-Series Cross-Validation

TimeSeriesSplit — test is always LATER than train

```
TimeSeriesSplit(n_splits=5, test_size=90)
```



Each fold's test block becomes part of the next fold's training data. That's why the training bar on the slide grows longer every row.

What happens inside each fold. The same four steps repeat five times:

1. Train a fresh model on only the training days. It has never seen anything after them.
2. Forecast the next 90 days.
3. Compare the forecast with what actually happened in those 90 days.
4. Record the error, e.g. the MAE.

⚠ NEVER use random KFold on time-series data — it leaks future information.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Classical Forecasting Methods

Three tools — increasing sophistication

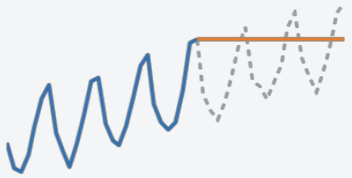
ETS Component/ What it tracks/ Smoothing weight
- Level/ Current baseline/ α (alpha
- Trend/ Direction and speed of change/ β (beta
- Seasonality/ Repeating pattern/ γ (gamma)

1

Naive

Predict the LAST value for everything.

- **Useful only as the bar floor.**

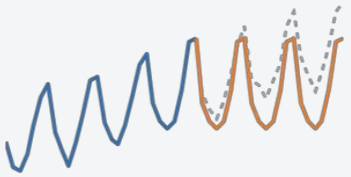


2

Seasonal Naive

Predict the value from ONE SEASON AGO (e.g., same-day-last-week).

- **Strong baseline when seasonality is stable, no trend.**



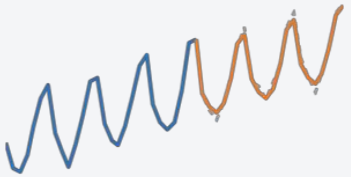
3

Exponential Smoothing (ETS)

Weighted average favouring recent values, plus trend and seasonality

Adaptive weighted average + trend + one seasonality.

- **The pragmatic baseline every forecaster should know.**



ETS is for forecasting: predicting future values of something measured over time, like next month's sales or next week's demand.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Exponential Smoothing (ETS) Variants



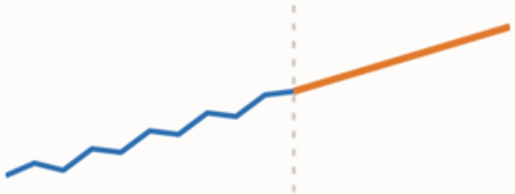
Simple (SES)

Level

Forecast: flat line at the current level

Use when: no trend, no season

Example: demand for a stable spare part



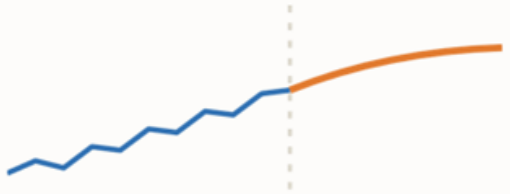
Holt's linear

Level · trend

Forecast: straight sloped line

Use when: steady growth or decline

Example: subscribers of a growing app



Damped trend

Level · fading trend

Forecast: slope that gradually flattens

Use when: growth should slow down

Example: sales of a maturing product



Holt-Winters

Level · trend · season

Forecast: trend plus repeating wave

Use when: trend and a regular cycle

Example: daily revenue with weekend peaks (the lab)

Copyright © 2026 Nanyang Technological University. All rights reserved.

Forecast Evaluation Metrics

Three standard ways to score a forecast

MASE: Mean Absolute Scaled Error. Error compared with the naive forecast (below 1 = better than naive) Error compared with the naive forecast. Below 1 means you beat naive; above 1 means you don't.

MAE

`mean(|y_true − y_pred|)`

Same units as the target. Intuitive.

Mean Absolute Error: Average size of the errors, in real units. Default, easy to explain

RMSE

$\text{sqrt}(\text{mean}((y_{true} - y_{pred})^2))$

Penalises LARGE errors more than small ones.

Root Mean Squared Error: Like MAE, but punishes big misses more. When large errors are especially costly (e.g. stock-outs).

MAPE

$\text{mean}(|y_{true} - y_{pred}| / |y_{true}|) \times 100\%$

Percentage. Scale-free. Watch out near zero.

Mean Absolute Percentage Error. Average error as a percentage ("off by 8%"). Comparing series of different sizes; fails when actual values are zero or near zero

Copyright © 2026 Nanyang Technological University. All rights reserved.

ML-with-Lags — The Modern Default

Time series → tabular regression with lag features

What are some features you can think of?

The trick

"For each day, ADD COLUMNS for past values (lag_1, lag_7, lag_365).
Now it's a tabular dataset — train any regression model on it."

Example feature table:

date	target	lag_1	lag_7	lag_365	dayofweek	month	...
2024-01-10	9200	8800	9100	8500	2	1	
2024-01-11	9400	9200	9300	8600	3	1	
2024-01-12	9700	9400	9500	8800	4	1	

Each row predicts its target from features that include past values. Standard regression — same gradient boosting from L04.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Bridge to Lesson 7

Marcus, after the Friday presentation:

"Great forecast. Now — if we collected DETAILED CUSTOMER BEHAVIOUR DATA
(every click, every page view, every cart event),
could you predict whether they'd complete checkout WHILE THEY'RE SHOPPING?"

Lesson 7 — Neural Networks & Deep Learning

Perceptron → MLP · gradient descent · backpropagation · PyTorch

End-to-end learning from raw sequences. The path to deep learning.

Before L07 — finish the L06 assignment (electricity demand + coffee shop).

Assignment Preview

Two domains · same toolkit

Part A — UK Electricity

- 2 years of daily demand data
- Strong annual + weekly cycles
- A1: STL decomposition
- A2: ETS-weekly baseline
- A3: ML-with-lags
- A4: which wins and why

Part B — Coffee Shop

- 18 months of daily customer count
- Weekly seasonality only
- B1: Seasonal Naive baseline
- B2: pick any method to beat B1 by 5%
- Smaller data, noisier signal

Copyright © 2026 Nanyang Technological University. All rights reserved.

Three Things to Remember

The practical wisdom beyond the algorithms

1

Time-series cross-validation is non-negotiable

Always use TimeSeriesSplit, never KFold. The future hasn't happened — training on it is leakage.

2

Lag features turn time series into tabular regression

Add lag_1, lag_7, lag_365, calendar columns. Train ANY regression model. Handles multi-seasonal data.

3

Evaluate on cross-validated windows, not a single split

A single window can give misleading results (e.g., the holiday window). Average across windows.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Method Comparison

When to use which forecasting tool

Traditional statistical forecasting techniques developed long before modern machine learning. They predict a series mainly from its own past values, using a fairly small set of mathematical rules

What about classical methods like ETS? They aren't "trained" like a random forest, but they still fit their settings (how much weight to give recent values, the trend, the seasonality) by minimising prediction error against actual past values. So they follow the same supervised idea.

Method	Best for	Pros	Cons
Naive	Bar floor; sanity check	Trivially simple	Almost always beaten
Seasonal Naive	Stable seasonal data	Captures seasonality	Misses trend; misses regime change
ETS	One seasonality + trend	Adaptive; well-tested	Single-seasonality only
ARIMA (Ext)	Stationary or differenced data	Theoretical foundation	Needs tuning; AutoARIMA helps
SARIMA (Ext)	Seasonal ARIMA	Bayesian, principled	Same single-seasonality limit
ML-with-lags	Multi-seasonal data	Same toolkit as L04	Recursive forecast errors compound
Prophet (Ext)	Multi-seasonal + holidays	Easy API	Heavier dependency

Prophet, a forecasting library made by Meta (Facebook), is best for data with several seasonal cycles plus holidays.

Predict Day 1 (small error)
→ use Day 1 prediction to predict Day 2 (error grows)
→ use Day 2 prediction to predict Day 3 (error grows more)

Copyright © 2026 Nanyang Technological University. All rights reserved.

Data situation/ Good choice
- Just need a benchmark/ Naive, seasonal naive
- One seasonal cycle + trend/ ETS (or SARIMA)
- Several seasonal cycles, holidays, extra factors/ ML-with-lags or Prophet

See You in L07

Neural Networks & Deep Learning — Perceptron → MLP → PyTorch

Before then: finish the L06 assignment (electricity demand + coffee shop)

Or dig deeper with ARIMA / SARIMA / Prophet in optional_extensions.ipynb

This table compares forecasting methods: what each is best for, its strengths and its weaknesses. “(Ext)” likely marks extension topics beyond the core lesson, and “L04” refers to an earlier lesson in your course (probably the one on random forests and gradient boosting).

Naive predicts the last value for everything. Its only real job is as a “bar floor”, the minimum benchmark; any real model must beat it. It’s trivially simple, but almost always beaten.

Seasonal naive predicts the value from one season ago (e.g. same day last week). It’s best for data with a stable repeating pattern and captures seasonality for free. Its cons:

Misses trend: it copies last season’s level, so it falls behind a rising series.
Misses regime change: a regime change is when the data’s behaviour shifts permanently, like a new competitor, a price change or COVID. Copying last season repeats patterns that no longer apply.

ETS is best for data with one seasonal cycle plus a trend. It’s adaptive, because it keeps updating level, trend and seasonality as new data arrives, and it’s well-tested after decades of reliable use. Its limitation is that it handles only one seasonal cycle.

ARIMA (AutoRegressive Integrated Moving Average) predicts from past values and past forecast errors. It’s best for stationary data: data whose average and spread stay roughly constant over time, with no trend or growing swings. If the data has a trend, ARIMA first differences it, working with the change between periods (Jan→Feb: +5, Feb→Mar: +3…) instead of raw values. Differencing usually removes the trend and makes the data stationary. That’s the “I” (Integrated) in ARIMA.

Its pro is a strong statistical foundation. Its con is that it needs tuning: you must choose three settings (p, d, q), which takes skill. AutoARIMA (e.g. pmdarima.auto_arima) tries many combinations and picks the best automatically.

SARIMA (Seasonal ARIMA) is ARIMA with an added seasonal part, and like ETS it handles only one seasonal cycle. One correction to the slide: it lists “Bayesian, principled” as SARIMA’s pro, but SARIMA isn’t normally a Bayesian method; it’s usually fitted with standard statistical estimation. “Principled” (well-grounded in statistical theory) is fair. The Bayesian label fits Prophet better, since it’s built on Bayesian modelling tools. It may be a slip on the slide, so worth checking with your instructor.

ML-with-lags turns forecasting into a normal machine learning problem: build lag features (yesterday’s value, last week’s value, rolling averages), then train a model like gradient boosting. It’s best for multi-seasonal data, since you can add features for hour of day, day of week, month and holidays all at once. Its pro is that it uses the same toolkit as L04 (random forests, boosting, pipelines) that you already know.

Its con is that recursive forecast errors compound. To forecast several days ahead, the model often predicts day 1, then uses that prediction as an input to predict day 2, and so on:

Predict Day 1 (small error)
  → use Day 1 prediction to predict Day 2 (error grows)
    → use Day 2 prediction to predict Day 3 (error grows more)

Each step builds on earlier guesses, so mistakes pile up the further ahead you forecast.

Prophet, a forecasting library made by Meta (Facebook), is best for data with several seasonal cycles plus holidays. You can give it a list of holidays like Chinese New Year or Hari Raya, and it models their effects. Its pro is an easy API, often just a few lines of code. Its con is that it’s a heavier dependency: an extra, fairly large library to install and maintain, with more moving parts than statsmodels.

The big picture. Read the table top to bottom as a progression:

Data situation

Good choice
Just need a benchmark

Naive, seasonal naive
One seasonal cycle + trend

ETS (or SARIMA)
Several seasonal cycles, holidays, extra factors

ML-with-lags or Prophet

The practical rule throughout: start simple, use the baselines as a bar, and only add complexity if it clearly lowers your validation error.